ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/344386051

HandBook Guidance on the programming of ABB YuMi IRB 14000

Method · September 2020

DOI: 10.13140/RG.2.2.12746.18881

CITATIONS
0

READS
7,604

1 author:



Adit Mak
Otto-von-Guericke University Magdeburg
2 PUBLICATIONS **0** CITATIONS

SEE PROFILE

All content following this page was uploaded by Adit Mak on 26 September 2020.

The user has requested enhancement of the downloaded file.

Otto-von-Guericke-University Magdeburg
Faculty of Electrical Engineering and Information Technology
Institute for Information Technology and Communication

HandBook

Guidance on the programming of ABB YuMi IRB 14000



submitted: May 31, 2020

by: Adit Makwana

Abstract

In short cycles, collaborative robots are able to add flexibility to assembly processes involving limited quantities of highly individualized items. By combining the unique ability of a person to adapt to change with the tireless endurance of a robot for precise, repetitive tasks, the assembly of many types of products on the same line can be automated. ABB has a new YuMi family member, the 7-axis YuMi, the smallest and most flexible collaborative robot, making it simpler to bring into production than ever. This handbook gives you a start with ABB YuMi IRB 14000 Robot Programming. This is a compilation of the information and the steps to be followed for programming from ABB's Operating Manual and open git repositories of KTH and ETH Zurich Universities. This demonstrates different ways to program YuMi robot, e.g. with FlexPendant, ABB RobotStudio, and ROS.

Keyword: ABB YuMi, FlexPendant, RobotStudio, ROS

Acknowledgement

It was a great opportunity to gain a lot of experience in field of robot programming. I would like to thank **Jun.-Prof. Dr.-Ing. Ingo Siegert** for his encouragement during this project and **Ing. Ralph Heinemann** for his exceptional laboratory support. I would also like to express my appreciation to Otto von Guericke University for giving me the opportunity to work with ABB YuMi Robot.

Contents

List of Tables v

List of Figures v

1 Introduction 1
- 1.1 IRB 14000 Robot . . . 1
  - 1.1.1 IRB 14000 integrated controller . . . 1
  - 1.1.2 Illustration . . . 1
  - 1.1.3 Axes and Co-ordinate Systems . . . 1
- 1.2 FlexPendant . . . 2
- 1.3 RobotWare . . . 3
- 1.4 RobotStudio . . . 3
- 1.5 ROS . . . 3

2 Programming with FlexPendant 4
- 2.1 Steps for creating a Program . . . 4
- 2.2 Other Relevant Information regarding FlexPendant . . . 5
  - 2.2.1 Creating a new Routine . . . 5
  - 2.2.2 Creating a new Module . . . 5
  - 2.2.3 Program Pointer and Motion Pointer . . . 5

3 Programming with RobotStudio 6
- 3.1 Building Station . . . 6
  - 3.1.1 Creating a solution with station and a robot controller . . . 6
  - 3.1.2 Creating a frame from three points . . . 6
  - 3.1.3 Creating a solid . . . 7
  - 3.1.4 Setting the local origin of an object . . . 7
- 3.2 Programming Robot . . . 8
  - 3.2.1 Workflow for programming a robot . . . 8
  - 3.2.2 Creating a workobject . . . 8
  - 3.2.3 Creating a Target . . . 9
  - 3.2.4 Creating an empty path . . . 9
  - 3.2.5 Synchronization . . . 10
- 3.3 Simulating Program . . . 10
- 3.4 Working Online . . . 11

4 Programming with ROS 12
- 4.1 Installation . . . 12
  - 4.1.1 Dependencies . . . 12
  - 4.1.2 Create a YuMi ROS Workspace . . . 12
  - 4.1.3 Clone the YuMi ROS Packages . . . 13
  - 4.1.4 Build Everything . . . 13
  - 4.1.5 Recent Changes in the YuMi Package . . . 13
- 4.2 Network Setup . . . 14
  - 4.2.1 Physical Connections . . . 14
  - 4.2.2 Network address . . . 14
- 4.3 Firmware Setup . . . 15
  - 4.3.1 Connecting to YuMi Controller . . . 15
  - 4.3.2 Updating YuMi Software and Firmware . . . 15
  - 4.3.3 Transfer ROS Modules to Controller . . . 16
  - 4.3.4 Setup YuMi Controller . . . 17

4.3.5 Setup YuMi ROS Interface through TouchPendant 20
4.3.6 Other Relevant Information Regarding YuMi 25
4.4 Control Interfaces 27
4.4.1 Position Control 27
4.4.2 Gripper Control 27
4.5 Live Robot 29
4.5.1 Starting RAPID scripts (Important) 29
4.5.2 Live Nodes 29
4.5.3 MoveIt! 29

5 Miscellaneous **31**
5.1 Calibration Error 31
5.2 Calibration Position 32
5.3 Operating Modes 33
5.3.1 Manual Mode 33
5.3.2 Automatic Mode 33

6 References **34**

List of Tables

1 Robot description . . . 1
2 Set Local origin . . . 7
3 Port description . . . 11
4 Tasks for ROS in RobotStudio . . . 18
5 List of Files to be assigned to Task for ROS . . . 18
6 Gripper Tasks for ROS in RobotStudio . . . 19
7 List of Gripper Files to be assigned to Task for ROS . . . 19
8 MoveIt! Changes in Yaml file . . . 29
9 Calibration Position (HOME) . . . 33

List of Figures

1 Robot Illustration . . . 1
2 Robot Axes . . . 2
3 Robot - Base Coordinate System . . . 2
4 Teach Pendant . . . 3
5 Workobject . . . 8
6 Ports on computer and robot . . . 11
7 Home Screen of Teach Pendant . . . 20
8 Quick_Access_Bar of Teach Pendant . . . 21
9 Initial_Active_List of Teach Pendant . . . 22
10 Main_Menu of Teach Pendant . . . 22
11 Program_Editor of Teach Pendant . . . 23
12 Task_Editor of Teach Pendant . . . 23
13 Debug_Sidebar of Teach Pendant . . . 24
14 Task_Windows of Teach Pendant . . . 24
15 Errors of Teach Pendant . . . 26
16 Calibration Scales . . . 31
17 Calibration Position . . . 32

1 Introduction

1.1 IRB 14000 Robot

The IRB 14000 robot is a two-armed industrial robot with integrated controller. Each arm has seven axes, which gives an extra degree of freedom compared to traditional 6-axis robots [4].

1.1.1 IRB 14000 integrated controller

The IRB 14000 integrated controller is based on the standard IRC5 controller, and contains all functions needed to move and control the robot.
The robot control software, RobotWare, supports every aspect of the robot system, such as motion control, development and execution of application programs, communication etc.

1.1.2 Illustration

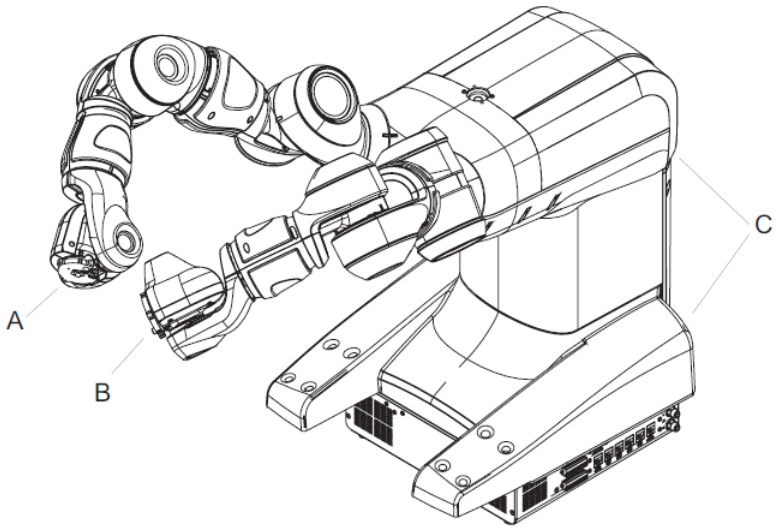


Figure 1: Robot Illustration

A	Right Arm
B	Left Arm
C	Integrated Controller

Table 1: Robot description

1.1.3 Axes and Co-ordinate Systems

The axes of the robot can be jogged manually using the joystick. Figure 2 shows the location and movement patterns for each axis.

The base coordinate system has its zero point in the base of the robot which is shown in Figure 3 .

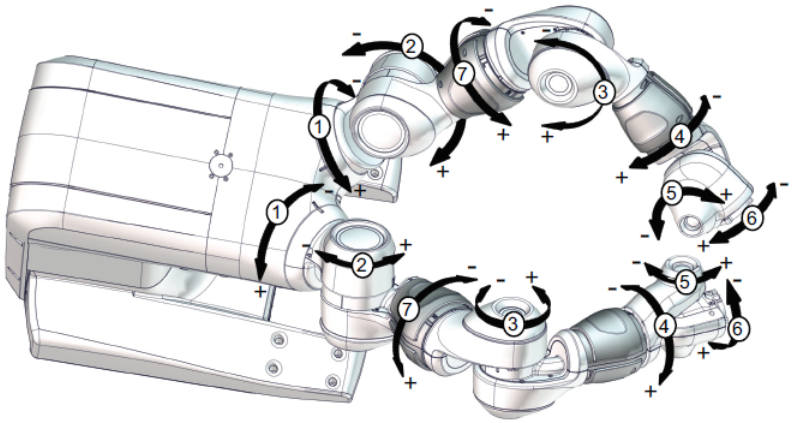


Figure 2: Robot Axes

When you are standing in front of the robot and jog in the base coordinate system, pulling the joystick towards you will move the robot along the X axis, while moving the joystick to the sides will move the robot along the Y axis. Twisting the joystick will move the robot along the Z axis.

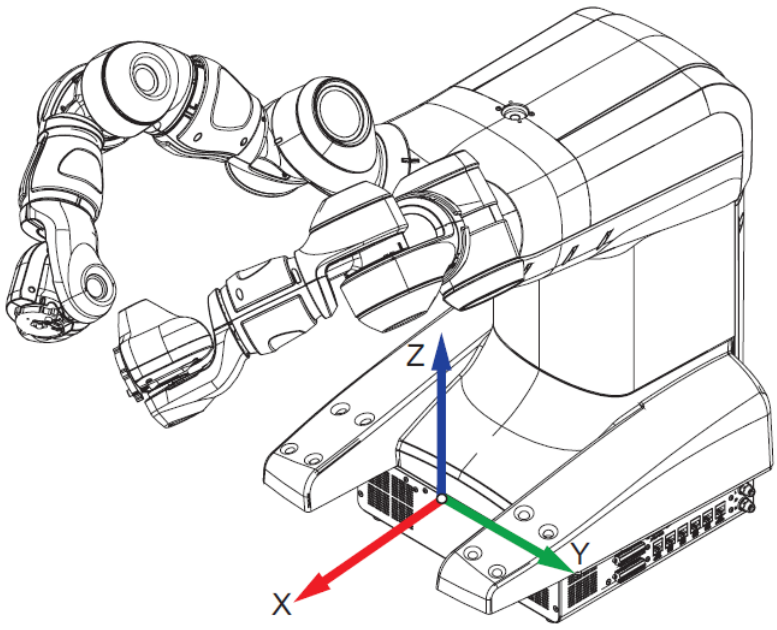


Figure 3: Robot - Base Coordinate System

1.2 FlexPendant

The FlexPendant is a hand held operator unit used to perform many of the tasks involved when operating a robot system, such as running programs, jogging the manipulator, modifying robot programs and so on.
The FlexPendant consists of both hardware and software and is a complete computer

in itself.



Figure 4: Teach Pendant

1.3 RobotWare

RobotWare is a family of software products from ABB Robotics. The products are designed to make you more productive and lower your cost of owning and operating a robot. ABB Robotics has invested many years into the development of these products and they represent knowledge and experience based on several thousands of robot installations.

1.4 RobotStudio

RobotStudio is an engineering tool for the configuration and programming of ABB robots, both real robots on the shop floor and virtual robots in a PC. To achieve true offline programming, RobotStudio utilizes ABB VirtualRobot™ Technology.
RobotStudio has adopted the Microsoft Office Fluent User Interface. The Office Fluent UI is also used in Microsoft Office. As in Office, the features of RobotStudio are designed in a workflow-oriented way.
With add-ins, RobotStudio can be extended and customized to suit the specific needs. Add-ins are developed using the RobotStudio SDK. With the SDK, it is also possible to develop custom SmartComponents which exceed the functionality provided by RobotStudio's base components.

1.5 ROS

The Robot Operating System (ROS) is a flexible framework for writing robot software. It is a collection of tools, libraries, and conventions that aim to simplify the task of creating complex and robust robot behavior across a wide variety of robotic platforms.

2 Programming with FlexPendant

The FlexPendant is best suited for modifying programs, such as positions and paths, while RobotStudio is preferred for more complex programming [3].

2.1 Steps for creating a Program

This section describes how to move the robot by creating a program with targets and motion. The steps has been shown for 1 arm of the robot but the same has to followed for other arm.

1. Creating a new program.
   (a) On the ABB menu, tap Program Editor.
   (b) Select the Robot Arm.
   (c) Tap Tasks and Programs.
   (d) Tap File, then New Program.
   (e) Continue by adding instructions, routines, or modules.
2. Adding Instructions
   (a) Jog the robot arm to desired location and keep it in desired position.
   (b) Tap Add instruction.
   (c) A category of instructions is displayed. A large number of instructions, divided into several categories, are available. The default category is Common, where the most common instructions are listed.
   (d) Tap the instruction you want to add. The instruction is added to the code.
   (e) For next instruction, follow the steps from 2a
3. Edit instruction Arguments
   (a) Tap the instruction to edit.
   (b) Tap Edit.
   (c) Tap Change Selected. Depending on the type of instruction, the arguments have different data types. Use the soft keyboard to change string values or proceed to the next steps for other data types or multiple argument instructions.
   (d) Tap the argument to be changed. A number of options are displayed.
   (e) Tap an existing data instance to select and then tap OK to complete.
4. Saving a program
   (a) Tap File and select Save Program As..
   (b) Use the suggested program name or tap ... to open the soft keyboard and enter a new name. Then tap OK.
5. Running a program
   (a) On the ABB menu, tap Program Editor.
   (b) Tap on the program step where you want to start, then tap Debug and then PP to Cursor. (More information in 2.2.3)
   (c) Press the Start button on the FlexPendant in downright corner.

2.2 Other Relevant Information regarding FlexPendant

2.2.1 Creating a new Routine

Routine is usually a set of data declarations followed by a set of instructions implementing a task.
This section details how to create a new routine.

1. On the ABB menu, tap Program Editor.
2. Select a Robot Arm.
3. Tap Routines.
4. Tap File, then New Routine. A new routine is created and displayed with default declaration values.
5. Tap ABC... and use the soft keyboard to enter the new routines' name. Then tap OK.
6. Select the Procedure routine.
7. Select module to add the routine to.

2.2.2 Creating a new Module

A set of data declarations followed by a set of routines. Modules can be saved, loaded and copied as files. Modules are divided into program modules and system modules.
Program modules can be loaded and unloaded during execution. System modules are used mainly for common system-specific data and routines, for example, an arcware system module that is common for all arc robots.
This section details how to create a new module.

1. On the ABB menu, tap Program Editor.
2. Select a Robot Arm.
3. Tap Modules.
4. Tap File, then New Module.
5. Tap ABC... and use the soft keyboard to enter the new modules' name. Then tap OK.
6. Select Program module to be created and tap OK.

2.2.3 Program Pointer and Motion Pointer

The Program Pointer (PP) indicates the instruction with which the program will start when you press any of the Start, Forward, or Backward buttons on the FlexPendant. Program execution continues from the instruction where the Program Pointer is. However, if the cursor is moved to another instruction when the program is stopped, the Program Pointer can be moved to the position of the cursor (or the cursor can be moved to the Program Pointer), and execution can be restarted from there. The Program Pointer is shown as a yellow arrow to the left of the program code in the Program Editor and Production Window.

The Motion Pointer (MP) indicates the instruction that the robot is currently executing. This is normally one or more instructions after the Program Pointer, as the system executes and calculates the robot path faster than the robot moves. The Motion Pointer is shown as a small robot to the left of the program code in the Program Editor and in the Production Window.

3 Programming with RobotStudio

3.1 Building Station

The following sections outline the workflow for building a new station. It also includes the prerequisites for creating and simulating robot programs [5]. The workflow includes:

- Creating a station with a system.
- Importing or creating the objects to work with.
- Optimizing the station layout by finding the best placement of robots and other equipment.

3.1.1 Creating a solution with station and a robot controller

Follow the steps below to set up a station that is ready for programming:

1. Click the File tab. The RobotStudio Backstage view appears, click New.
2. Under Stations, click Solution with Empty Station.
3. Import a robot model.
4. Import a tool and attach the tools to the robot.
5. Import a positioner or a track (if applicable).
6. Add work piece and attach work pieces to positioners.
7. Test reachability.
8. Create a station with an existing system layout.
9. Click From Layout to bring up the first page of the wizard.
10. In the Name box, enter the name of the system. The location of the system will be displayed in the Location box.
11. In the RobotWare list, select the version of RobotWare you want to use.
12. Click Next.
13. In the Mechanisms box, select the mechanisms that you want to include in the system.
14. Click Next. The System Option page appears.
15. On the System Option page, you have the option to align Task Frame(s) with the corresponding Base Frame(s).
    - For MultiMove Independent system, select the check box to align task frame with base frame for each robot.
    - For MultiMove Coordinated system, select the robot from the drop down list and select the check box to align task frame with base frame for the selected robot.
16. Verify the summary and then click Finish.

3.1.2 Creating a frame from three points

1. Click Frame from Three points to bring up a dialog box.
2. Decide how you want to specify the frame.
3. If you select Position:

(a) Enter the Position for the object.
(b) Enter the Point on X axis for the object.
(c) Enter the Point on X-Y plane for the object.
(d) Click Create.

4. If you select Three Point:

(a) Enter the First Point on X axis for the object. This is the point closest to the frame's origin.
(b) Enter the Second Point on X axis for the object. This is the point further away in the positive X direction.
(c) Enter the Point on Y axis for the object.
(d) Click Create.

3.1.3 Creating a solid

With the create solids commands you can create and build models of objects you do not have CAD files or libraries for. With the create solids commands you create primitve solid bodies; these can later be combined to more complex bodies.

1. Click Solid and then click the type of solid you want to create to bring up a dialog box.
2. Enter requested values in the dialog box and click Create.

3.1.4 Setting the local origin of an object

Each object has a coordinate system of its own called local coordinate system in which the object dimensions are defined. When the object's position is referred from other coordinate system, it is the origin of this coordinate system that is used. With the Set Local Origin command you reposition the object's local coordinate system, not the object itself.

1. If the object you want to modify is a library component, first disconnect it from the library.
2. In the Layout browser or the graphics window, select the part to modify.
3. Click Set Local Origin to bring up a dialog box.
4. In the Set Local Origin dialog box, select the reference coordinate system you want to use:

Move	**Select**
relative to the part's current local coordinate system	Local
relative to the coordinate system of its parent	Parent
absolute in the coordinate system of the station	World
relative to a user-defined coordinate system	UCS

Table 2: Set Local origin

5. In the Position X, Y, Z boxes, either type the new position or, click to select it in one of the value boxes and then click the point in the Graphics window.
6. Type the Orientation.
7. Click Apply.

3.2 Programming Robot

3.2.1 Workflow for programming a robot

Going through the workflow from start to finish is recommended, even if it is possible to work in other sequences as well.

The sequence below describes the workflow for programming a robot to perform the task you require.

1. Create targets and paths the robot requires to perform the work tasks.
   (a) Create a curve to match your required shape. Then use the Create path from curve command to generate a path, complete with targets, along the shape you have created.
   (b) Create targets at the requested positions, then create a path and insert the created targets into it. (More details are mentioned in 3.2.3 and 3.2.4)
2. Check targets are oriented in the most efficient way for the tasks to be performed. If not, reorient the targets.
3. Check that the robot and tool reach all targets in the path.
4. Synchronize the program to the virtual controller.
5. Perform text-based editing.
6. Collision detection.
7. Test the program

3.2.2 Creating a workobject

A workobject is a coordinate system used to describe the position of a work piece. The workobject consists of two frames: a user frame and an object frame. All programed positions will be related to the object frame, which is related to the user frame, which is related to the world coordinate system.

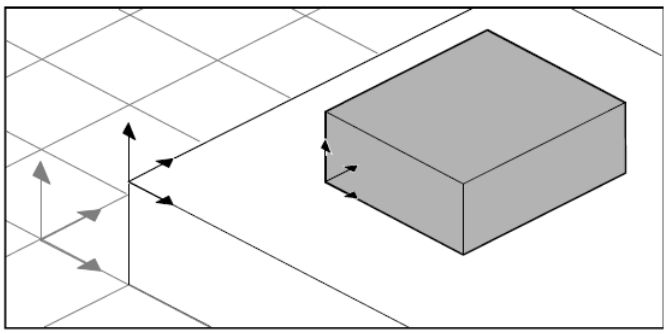


Figure 5: Workobject

1. On the Home tab, in the Path Programming group, click Other and select Create Workobject. The Create Workobject dialog box appears.
2. In the Misc Data group, enter the values for the new workobject.
3. In the User Frame group, do one of the following:
   (a) Set the position of the user frame by entering values for the Position x,y,z and the Rotation rx, ry, rz for the workobject by clicking in the Values box.

(b) Select the user frame by using the Frame by points dialog box.

4. In the Object Frame group you can reposition the object frame relative to the user frame by doing any of the following:
   (a) Set the position of the object frame by selecting values for Position x, y, z by clicking in the Values box.
   (b) For the Rotation rx, ry, rz, select RPY (Euler XYX) or Quaternion, and enter the rotation values in the Values dialog box.
   (c) Select the object frame by using the Frame by points dialog box.
5. In the Sync Properties group, enter the values for the new workobject.
6. Click Create. The workobject will be created and displayed under the Targets node under the robot node in the Paths&Targets browser.

3.2.3 Creating a Target

Create a new target manually either by entering the position for the target in the Create Target dialog box or by clicking in the graphics window. The target will be created in the active workobject.

1. In the Layout browser, select the workobject in which you want to create the target.
2. Click Create Target to bring up a dialog box.
3. Select the Reference coordinate system you want to use to position the target. In the Points box, click Add New and then click the desired position in the graphics window to set the position of the target. You can also enter the values in the Coordinates boxes and click Add.
4. Enter the Orientation for the target. A preliminary cross will be shown in the graphics window at the selected position. Adjust the position, if necessary. To create the target, click Create.
5. If you want to change the workobject for which the target is to be created, expand the Create Target dialog box by clicking the More button. In the WorkObject list, select the workobject in which you want to create the target.
6. If you want to change the target name from the default name, expand the Create Target dialog box by clicking the More button and entering the new name in the Target name box.
7. Click Create. The target will appear in the browser and in the graphics window.

3.2.4 Creating an empty path

A path is a sequence of targets with move instructions that the robot follows. An empty path will be created in the active task.

1. In the Paths&Targets browser, select the folder in which you want to create the path.
2. Click Empty Path.
3. To set the correct motion properties for the targets, select the active process in the Change Active Process box in the Elements toolbar.
4. If the active template is set to MoveAbsJoint, then:

(a) A target that is dragged into a path will be converted into a jointtarget.
(b) Joint targets and their instructions can only use wobj0 and tool0.
(c) One target can not be used as different types, for example, MoveJoint, but must be deleted and re-created.
(d) When the target has been synchronized with the virtual controller, the joint target values will be calculated and inserted in the RAPID program.

5. It is possible to call a procedure from another procedure. In the Paths & Targets browser, click Paths & Procedures to view the folder structure. You can move procedures between folders using a drag-and-drop operation.

3.2.5 Synchronization

To synchronize is to make sure that the RAPID program in the system running on the virtual controller corresponds to the programs in RobotStudio. You can synchronize both from RobotStudio to the virtual controller and from the virtual controller to RobotStudio.

When to synchronize the station to the VC: Synchronizing the station to the VC updates the RAPID program of the virtual controller with the latest changes in the station. This is useful to do before:

- Performing a simulation.
- Saving a program to files on the PC.
- Copying or loading RobotWare systems

When to synchronize the VC to the station: Synchronizing the VC to the station creates paths, targets and instructions that correspond to the RAPID program in the system running on the virtual controller. This is useful to do when you have:

- Started a new virtual controller which system contains existing programs.
- Loaded a program from a file.
- Text-edited the program.

3.3 Simulating Program

This section describes how to simulate and validate robot programs.

1. Play simulations: Simulations run entire robot programs on a virtual controller.
2. Collision detection: displays and logs collisions and near-misses for specified objects in the station. Normally used during simulation of robot programs, it can also be used when building the station.
3. Event handling: Events can be used to connect an action to a trigger. For example, you can attach one object to another when they collide or a signal is set.
4. I/O Simulation: In simulations I/O signals are normally set either by the robot program or by events. With the I/O simulator you can set signals manually, which provides a quick test of specific conditions.
5. Simulation Monitoring: With the simulation monitoring functions you enhance the simulation by adding traces along the TCP movements or alerts triggered by defined speeds or motions.
6. Process time measurement: With the process timer you measure the time for a process to complete.

3.4 Working Online

In general there are two ways of physically connecting a PC to the controller, to the service port or to the factory network port.

The **service port** is intended for service engineers and programmers connecting directly to the controller with a PC. The service port is configured with a fixed IP-address, which is the same for all controllers and cannot be changed, and has a DHCP server that automatically assigns an IP-address to the connected PC.

The **factory network port** is intended for connecting the controller to a network. The network settings can be configured with any IP-address, typically provided by the network administrator.

The illustration below shows the two main ports on the computer unit DSQC 639, the service port and the LAN port.

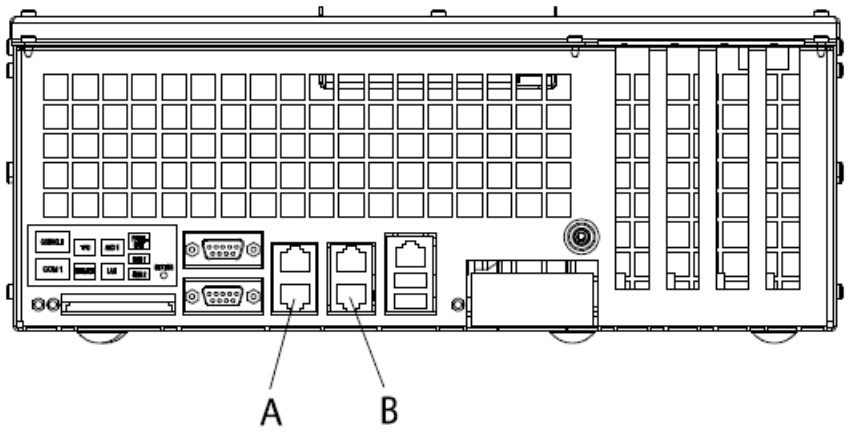


Figure 6: Ports on computer and robot

A	Service port on the computer unit (connected to the service port on the controller front through a cable).
B	LAN port on the computer unit (connects to factory network).

Table 3: Port description

Note: To connect Robot to the RobotStudio in PC please refer 4.3.1.

4 Programming with ROS

4.1 Installation

4.1.1 Dependencies

This package is tested with **Ubuntu 16.04 64 bits** and **ROS Kinetic Kame**. To install the dependencies for this package run the following commands in a terminal [2] [1]:

```
sudo apt-get install \
  python-pip \
  protobuf-compiler \
  protobuf-c-compiler \
  ros-$ROS_DISTRO-control-toolbox \
  ros-$ROS_DISTRO-controller-interface \
  ros-$ROS_DISTRO-controller-manager \
  ros-$ROS_DISTRO-effort-controllers \
  ros-$ROS_DISTRO-force-torque-sensor-controller \
  ros-$ROS_DISTRO-gazebo-ros-control \
  ros-$ROS_DISTRO-joint-limits-interface \
  ros-$ROS_DISTRO-joint-state-publisher \
  ros-$ROS_DISTRO-joint-state-controller \
  ros-$ROS_DISTRO-joint-trajectory-controller \
  ros-$ROS_DISTRO-moveit-commander \
  ros-$ROS_DISTRO-moveit-core \
  ros-$ROS_DISTRO-moveit-planners \
  ros-$ROS_DISTRO-moveit-ros-move-group \
  ros-$ROS_DISTRO-moveit-ros-planning \
  ros-$ROS_DISTRO-moveit-ros-visualization \
  ros-$ROS_DISTRO-moveit-simple-controller-manager \
  ros-$ROS_DISTRO-position-controllers \
  ros-$ROS_DISTRO-rqt-joint-trajectory-controller \
  ros-$ROS_DISTRO-transmission-interface \
  ros-$ROS_DISTRO-velocity-controllers \
  ros-$ROS_DISTRO-moveit
pip install --user pyftpdlib
pip install --user --upgrade pyassimp
```

4.1.2 Create a YuMi ROS Workspace

```
mkdir -p yumi_depends_ws/src
cd yumi_depends_ws/src
catkin_init_workspace
cd ..
catkin_make
echo "export YUMI_WS='$(pwd)'" >> ~/.bashrc
echo "source $(pwd)/devel/setup.bash" >> ~/.bashrc
```

4.1.3 Clone the YuMi ROS Packages

```
bash
cd $YUMI_WS/src
git clone \
https://github.com/OrebroUniversity/industrial_core.git\
sed -i '4 i add_compile_options(-std=c++11)'\
industrial_core/industrial_trajectory_filters/\
CMakeLists.txt
git clone \
https://github.com/ros-industrial/abb.git
rm -rf abb/abb_irb4400_support abb/abb_resources
git clone https://github.com/kth-ros-pkg/yumi.git
git clone \
https://github.com/tu-darmstadt-ros-pkg/hector_models.git
git clone https://github.com/kth-ros-pkg/optoforce.git
git clone \
https://github.com/kth-ros-pkg/optoforce_ros.git
git clone \
https://github.com/kth-ros-pkg/etherdaq_ros.git
git clone \
https://github.com/kth-ros-pkg/force_torque_tools
```

4.1.4 Build Everything

```
bash
cd $YUMI_WS
catkin_make -DCMAKE_BUILD_TYPE=Release
```

4.1.5 Recent Changes in the YuMi Package

Warning: this will erase any changes you have made to the original yumi package. Hence, save any changes made to the repo before running the commands shown below.

```
cd $YUMI_WS
rm -rf build/ devel/
cd src/
rm -rf yumi
git clone https://github.com/kth-ros-pkg/yumi.git
cd yumi
git checkout origin/<branch_name>
cd $YUMI_WS
catkin_make -DCMAKE_BUILD_TYPE=Release
```

4.2 Network Setup

4.2.1 Physical Connections

The first thing that you should do to be able to control YuMi, is to connect an Ethernet cable from your computer (or network switch), to the robot's **XP23** port.

4.2.2 Network address

YuMi requires that you connect from a single, predetermined IP address **192.168.125.50**. Hence, we need to assign that address to the interface that you will use to connect to the robot.
In this tutorial, represents the network interface through which you want to connect to YuMi (e.g. wlan0, eth0, enp5s0f1, etc). To find out the names of all your system interfaces by running the command ifconfig without any parameters in a terminal.

To set up the IP temporarily, run:

```
sudo ifconfig <interface> 192.168.125.50
```

On the other hand, if you want to set the IP from startup, such that you don't need to run the command each time, add the following lines to your */etc/network/interface* file:

```
auto <interface>
iface <interface> inet static
address 192.168.125.50
netmask 255.255.255.0
```

If you chose to set up the IP address from startup, after saving the file you'll also need to restart the network interface by running the following commands on a terminal:

```
sudo ifdown <interface>
sudo ifup <interface>
```

4.3 Firmware Setup

4.3.1 Connecting to YuMi Controller

Below are instructions to setup the YuMi controller in RobotStudio to allow YuMi to be able to communicate with ROS.

1. Turn on YuMi and wait for the home screen to come up on the FlexPendant
2. Connect YuMi to your computer using an Ethernet cable from the XP23 Service Port
3. Open RobotStudio 6.0 or newer
4. Click Online, then click Add Controller
5. In the "Remote Controller" box, type: 192.168.125.1
6. Click Add, controller should appear above
7. Click the controller, then click OK

You are now connected to the YuMi controller.

4.3.2 Updating YuMi Software and Firmware

To make sure the steps below will work properly, you need to check that the most up to date software has been loaded onto YuMi and the Smart Gripper package has been added as well. The steps to check this is listed below.

1. Turn on YuMi and wait for the home screen to come up on the FlexPendant
2. Connect YuMi to your computer using an Ethernet cable from the XP23 Service Port
3. Connect YuMi to your computer using an Ethernet cable from the XP23 Service Port
4. Click Online, then click Add Controller
5. In the "Remote Controller" box, type: 192.168.125.1
6. Click Add, controller should appear above
7. Click the controller, then click OK
8. Request Write Access from YuMi (more information below)
9. Click the Grant button in the popup window on the FlexPendant screen

Point 8 - $ControllerTab > AccessGroup > RequestWriteAccess$

Now the computer running RobotStudio has access to the YuMi controller. First you will check which software version is installed and install the Smart Gripper add-on package if you have not done so yet. To do this, do the following:

1. Go to the _Add-In_s tab found at the top of the RobotStudio window
2. Click the RobotWare button if it is not currently displaying
3. Check which version is installed and make sure it is the newest version
4. Then click the RobotWare Add-Ins button and find the SmartGripper package
5. If the SmartGripper package is not installed, then install it

6. Go to the Controller tab
7. Click the Installation Manager button in the Configuration button group
8. In the new window, make sure the Network tab is active and double click controller in the list
9. Click next on the bottom right of the window
10. The RobotWare and SmartGripper need to be added in this window
11. To add the packages, click Add... and add both the RobotWare and SmartGripper packages
12. Once this is done, all the neccessary packages have been installed then click Next
13. A license should be displayed in the current window, click Next again
14. The following controller options need to be enabled:
    (a) PC Interface (616-1) found at: $SystemOptions > Communication > PCInterface$
    (b) Multitasking (623-1) found at: $SystemOptions > EngineeringTools > Multitasking$
15. Once this is done, click Next then Apply

Point 3 - This repo was made based on the RobotWare version 6.03.02
Point 14b - This option should be enabled by default since YuMi has two arms that collaborate

YuMi should now have the required software, packages, and controller settings neccessary to communicate with an external computer.

4.3.3 Transfer ROS Modules to Controller

Below are instructions on how to move the ROS modules in this YuMi repo to the robot controller. The ROS module files can be found in *yumi_hw/rapid/*

1. Open up File Transfer (more information below)
2. The right side shows the files on YuMi, the left side shows the RobotStudio folder in Documents on your computer
3. On the YuMi files side (right side), create a folder called ROS and go into that folder (more information below)
4. On the PC Explorer side (left side), navigate to the ROS module files that are included in this repo
5. Highlight all of the ROS modules and system files, then click the right arrow button in the middle

Part 1 - $ControllerTab > ControllerToolsGroup > FileTransferPart2 - Rightclick > NewFolder$

All the ROS module and system files should now be in a folder called ROS in the Home directory of the YuMi controller.

NOTE: To make sure that the PC Explorer side of the window (left side) is up to date with anyrecent changes made to the files that will be uploaded to YuMi, click the refresh button located on the top of the PC Explorer box.

4.3.4 Setup YuMi Controller

Below are the instructions for adding the ROS modules in this YuMi repo to the robot controller's configuration.

Note: You will need write access to YuMi from RobotStudio in order to do the steps below. To get write access, on the Controller tab in RobotStudio click Request Write Access located in the Access button group. On the FlexPendant, a window will appear saying that a user is requesting right access. Click the Grant button in the pop window on the FlexPendant screen and then you will be able to do the steps below.

1. Open up the Configuration Editor on the Controller tab (more information below)
2. Click "Task" on the left panel of the Configuration Editor Screen
3. Right click in the task window and choose "New Task..."
4. Add the first task in the Tasks table below, leave unnamed inputs blank
5. Click Ok and repeat until all tasks are finished
6. Click "Automatic Loading of Modules" from the left panel of the screen
7. Right click in the modules window and choose "New Automatic loading of modules..."
8. Add the first module in the Modules table below, leave unnamed inputs blank
9. Click Ok and repeat until all modules are finished
10. Ensure all tasks and modules have been inputted correctly
11. P-Start the controller to put changes into effect (more information below)

Point 1 - $ControllerTab > ConfigurationGroup > ConfigurationEditorDropdown > Controller$
Point 11 - $ControllerTab > ControllerToolsGroup > RestartDropdown > ResetRAPID(P - Start)$

Note: If you are using an existing YuMi, the controller will reset and you will need to wait until the FlexPendant is back to the home screen

Tasks:

Keep all unnamed inputs in the table above as the default value

All the additional tasks are added as semistatic. This means these tasks are meant to run in the backgroun and are not supposed to be stopped or restarted during normal operation of YuMi.

TASK	TYPE	TRUST LEVEL	MECHANICAL GROUP
ROS_MotionServer_Left	SEMISTATIC	SysStop	rob_l
ROS_MotionServer_Right	SEMISTATIC	SysStop	rob_r
ROS_StateServer_Left	SEMISTATIC	No Safety	rob_l
ROS_StateServer_Right	SEMISTATIC	No Safety	rob_r

Table 4: Tasks for ROS in RobotStudio

After creating the tasks shown in the table above, restart the YuMi via the FlexPendant. When it finishes restarting, open RobotStudio, and add the following scripts (all of them located in *yumi_hw/rapid/*) to each of the tasks specified:

File	Tasks
ABB_driver/ROS_motion_left.mod	T_ROB_L
ABB_driver/ROS_motion_right.mod	T_ROB_R
ABB_driver/ROS_motionServer_left.mod	ROS_MotionServer_L
ABB_driver/ROS_motionServer_right.mod	ROS_MotionServer_R
ABB_driver/ROS_stateServer_left.mod	ROS_StateServer_L
ABB_driver/ROS_stateServer_right.mod	ROS_StateServer_R
ABB_driver/ROS_common.sys	All RWS tasks
ABB_driver/ROS_messages.sys	All RWS tasks
ABB_driver/ROS_socket.sys	All RWS tasks

Table 5: List of Files to be assigned to Task for ROS

NOTE: If any changes are made the RAPID modules after these modules and tasks have been loaded to the controller, these changes will be overwritten each time YuMi is turned on then off. This is because changes made to the RAPID modules in RobotStudio does not also change the files located in memory. Due to this, adding the automatic loading of modules will overwrite whatever files are located in their assigned tasks when YuMi is started up.

Grippers:

Task:

TASK	TYPE
T_GRIP_State	NORMAL
T_GRIP_Motion	NORMAL
T_GRIP_Motion_L	NORMAL
T_GRIP_Motion_R	NORMAL

Table 6: Gripper Tasks for ROS in RobotStudio

Modules:

File	Tasks
Grippers/Gripper_motionServer.mod	T_GRIP_Motion
Grippers/Gripper_stateServer.mod	T_GRIP_State
Grippers/GripperMotion_left.mod	T_GRIP_Motion_L
Grippers/HandDriver_left.sys	T_GRIP_Motion_L
Grippers/GripperMotion_right.mod	T_GRIP_Motion_R
Grippers/HandDriver_right.sys	T_GRIP_Motion_R
Grippers/ROS_common.sys	All T_GRIP_XXX tasks
Grippers/ROS_messages.sys	All T_GRIP_XXX tasks
Grippers/ROS_socket.sys	All T_GRIP_XXX tasks

Table 7: List of Gripper Files to be assigned to Task for ROS

4.3.5 Setup YuMi ROS Interface through TouchPendant

Once the YuMi has been turned on, the TouchPendant screen should be similar to Figure 7 below once the YuMi OS has been loaded. There are four buttons on this screen.

1. The menu button is the button on the top left.
2. On the right of menu button shows any message have been sent to the TouchPendant (messages are generally only sent from RAPID code and currently only show the user the current state of each RAPID task).
3. The wide button on the top of the screen shows the event log, displaying any high-level operations that have been performed recently along with showing an errors that have popped up.
4. The button on the bottom right is the quick access button, which allows for easy access to some of the basic functionality. This includes turning the motors on and off, setting manual or automatic mode, changing the max motor speeds, and a few others.



Figure 7: Home Screen of Teach Pendant

Set All Tasks to Active

When the YuMi is first turned on, the state and motion server tasks for each arm are started once the YuMi OS has been loaded on the TouchPendant. Before continuing below, ensure that the state and motion server tasks are in a running state and ensure that only the motion tasks and set to active. To do this, click the quick access button



Figure 8: Quick_Access_Bar of Teach Pendant

on the bottom right of the home screen. Once that button is pushed, a sidebar should appear on the right side of the screen as shown below in Figure 8

Click the bottom button in the quick access menu sidebar (a hollow circle branching to three smaller filled circles) to bring up the active tasks list. The screen should be the same as Figure 9 below, with the tasks T_ROB_L and T_ROB_R checked, and the rest unchecked with a "Go" status. If the active task list screen does not look the same as shown below, the please follow the instructions in the section State or Motion Server Task Error below and follow the instructions listed there before continuing.

Set Program Pointer to Main Function for All Tasks

From the home screen, click the menu button on the top left, then click Program Editor (menu screen shown in Figure 10. The screen will show the list of tasks that are on YuMi, the list should look the same as shown in Figure 11.

Click on one of the motion tasks, either T_ROB_L or T_ROB_R (Figure 13 shows the screen if the T_ROB_L task is chosen), and the RAPID code for that motion task should be displayed as shown in Figure 12. To set the program pointer to the main function, first click the Debug button showed on the bottom menu bar. A sidebar should appear as shown in Figure 13. Click the button labeled PP to Main to set the program point to main for the task displayed.

This process will need to be repeated for the other motion task (T_ROB_L and T_ROB_R). Push the menu button on the top left, then press the Program Editor button again. Click a the other motion task from what was chosen in the previous step, then reset the program pointer to main as described above. Once this done, two windows should be open, as shown on the bottom left of Figure 14 below with the two button labeled "T_ROB_L ROS_mot..." and "T_ROB_R ROS_mot...".

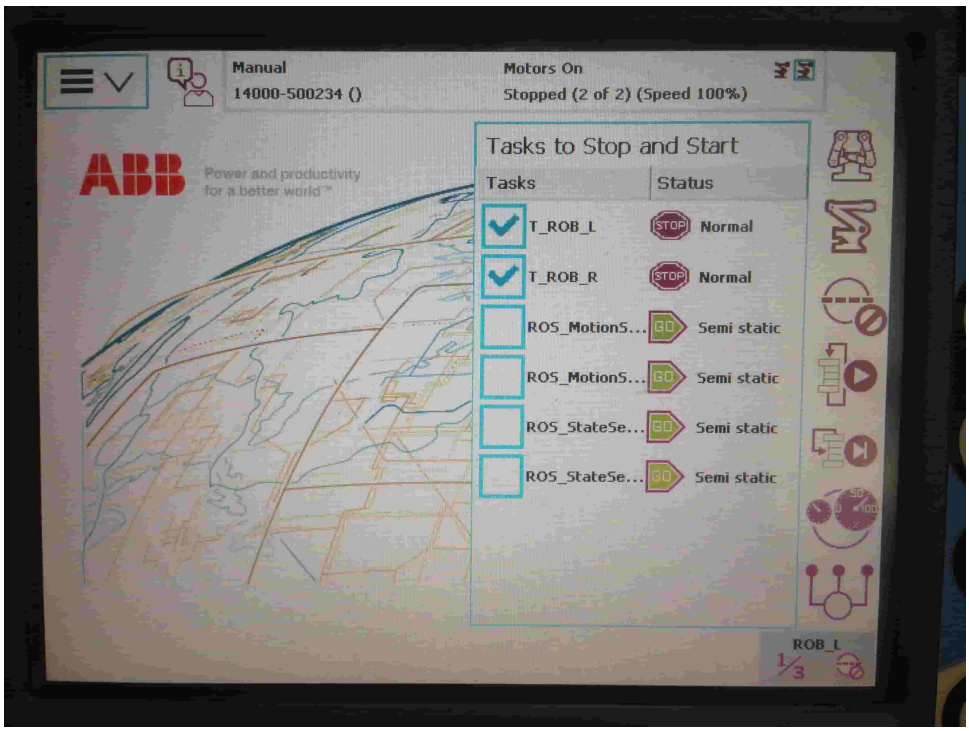


Figure 9: Initial_Active_List of Teach Pendant

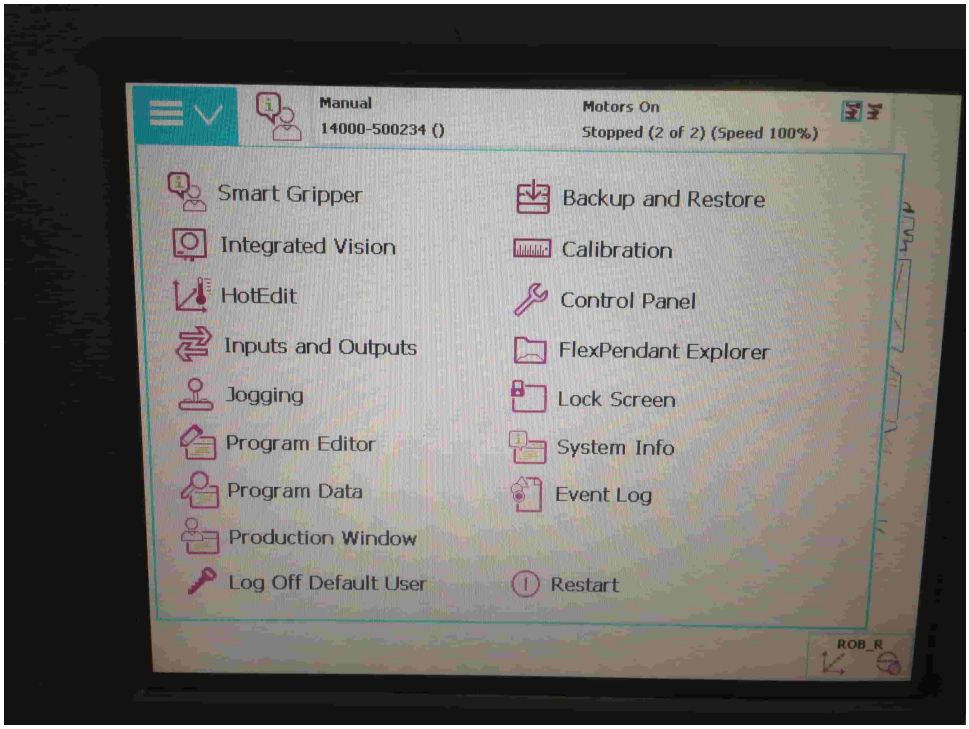


Figure 10: Main_Menu of Teach Pendant

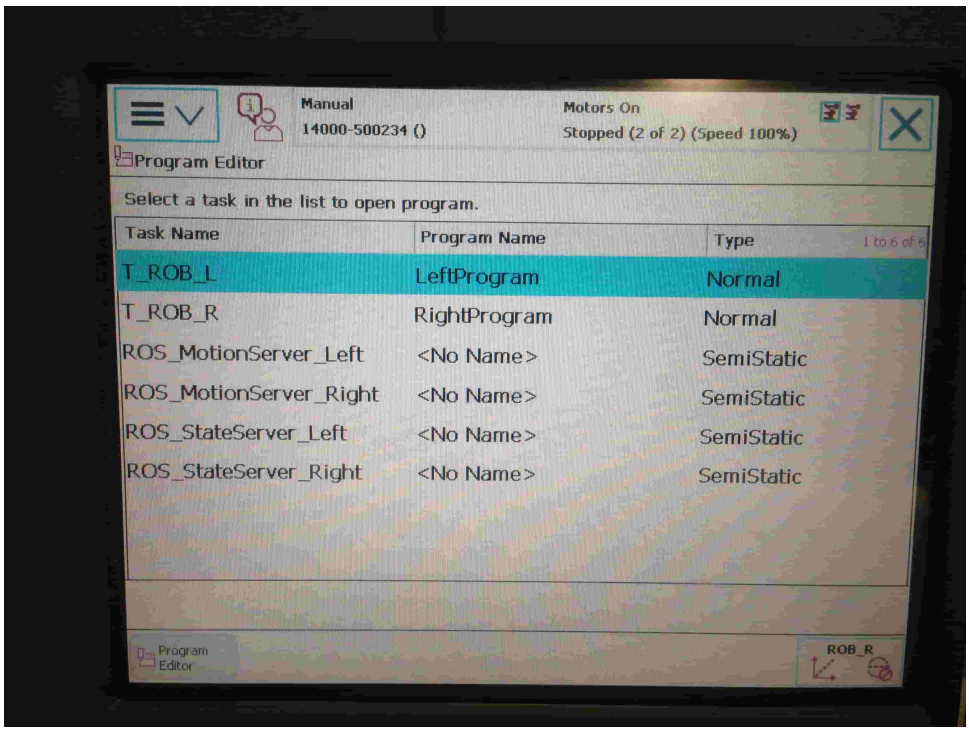


Figure 11: Program_Editor of Teach Pendant

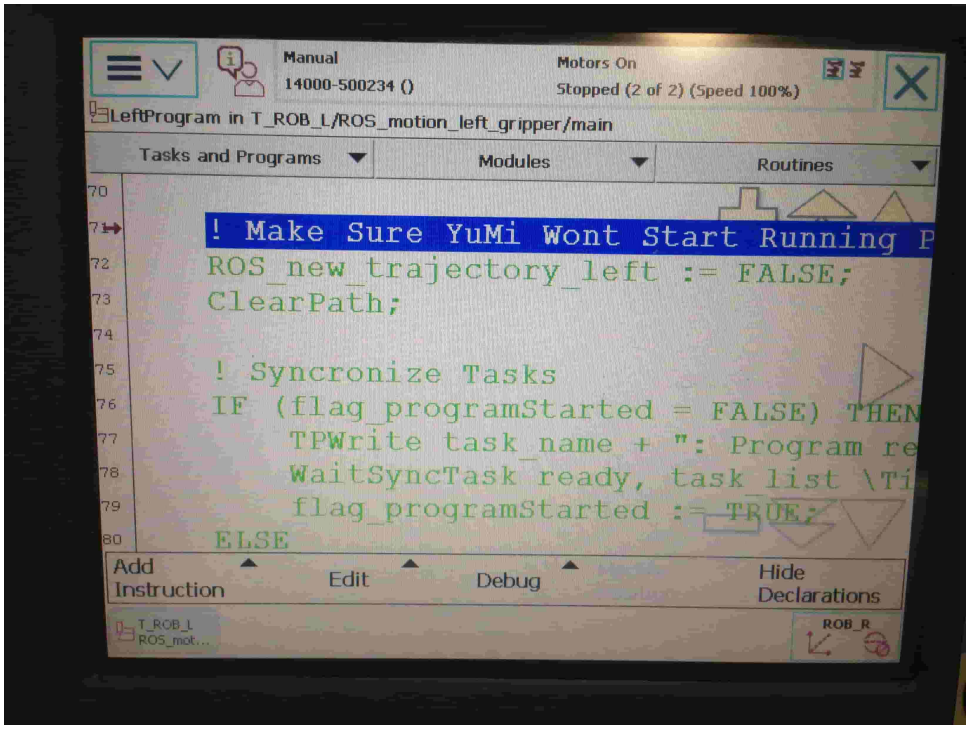


Figure 12: Task_Editor of Teach Pendant

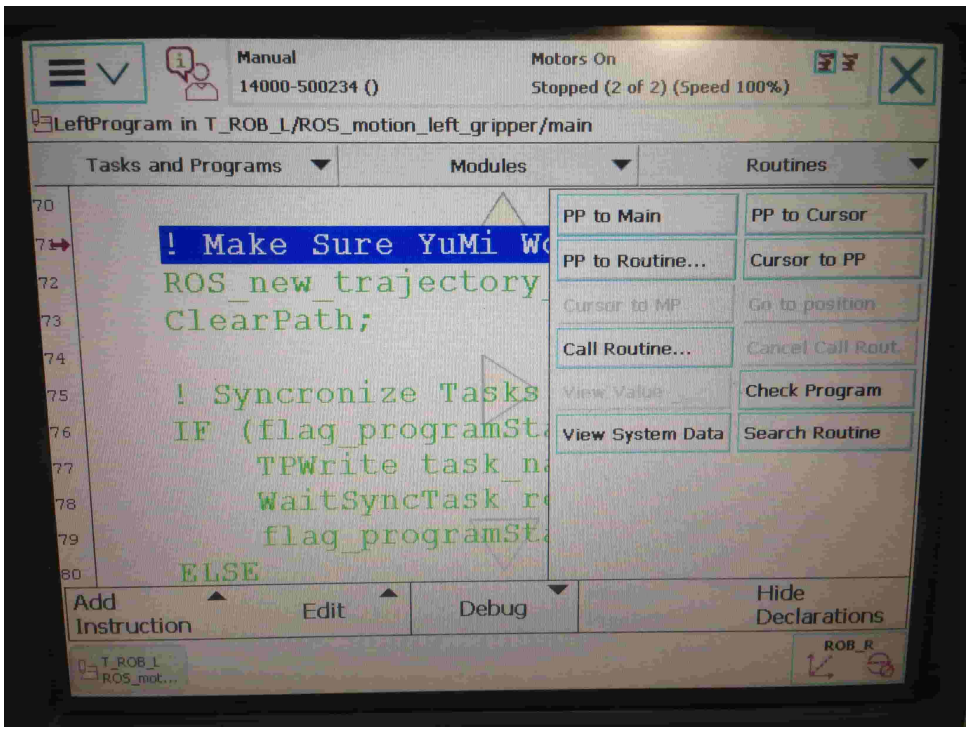


Figure 13: Debug_Sidebar of Teach Pendant

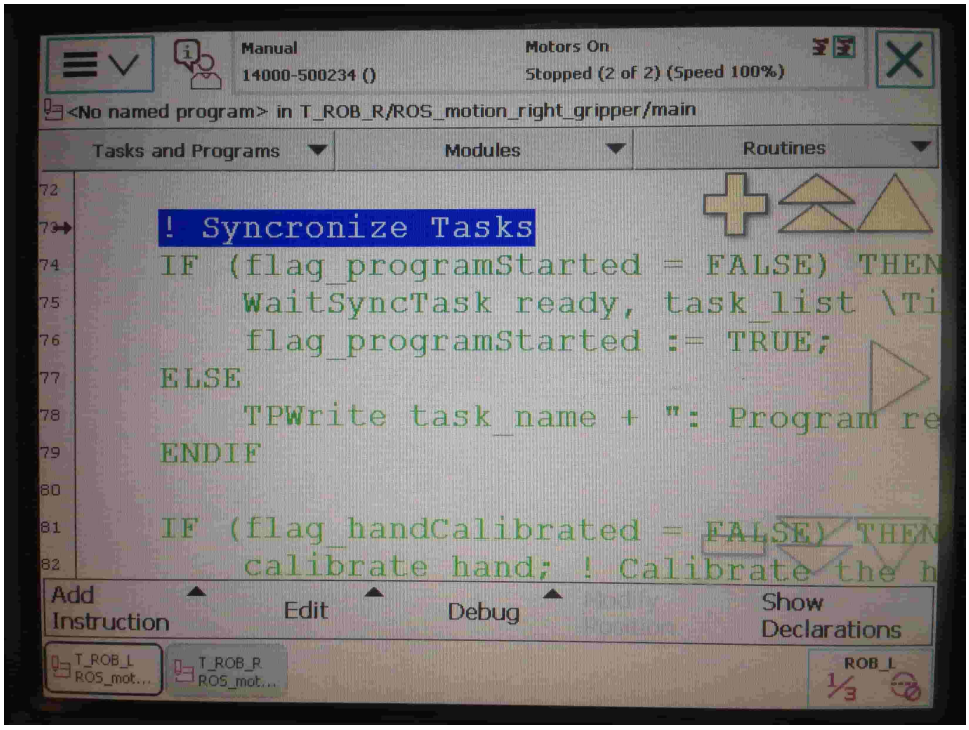


Figure 14: Task_Windows of Teach Pendant

Running the Tasks

Once all of the operations listed above have been completed, then the motion tasks can be executed. To do this, press the play button located on the bottom right of the TouchPendant (not the screen). The two grippers (if YuMi was set up for two grippers, otherwise ensure YuMi has been setup correctly as described on the YuMi ROS Integration wiki page) will go through a calibration process of closing completely, then moving to 10mm and back to the closed position to indicate that grippers have been calibrated.

Occasionally, one or both of the grippers will not close completely before calibrating, thus setting the zero position incorrectly. If this occurs, click the stop button on the bottom right of the TouchPendant (not the screen), and reset the motion tasks program pointers to the main function as was done in the previous section, then press the play button on the bottom right of the TouchPendant (not the screen).

If one of the grippers does not move at all, then most likely the tasks program pointer was not set to the main function. Stop the motion tasks, set the motion tasks program pointers to the main function, then start the tasks again.

Stopping and Re-Running the Tasks

To stop the motion tasks, simply click the stop button on the bottom right of the TouchPendant (not the screen). If the user would like to re-run the tasks, first the motion tasks program pointer to their main functions, as described above, then click the play button on the bottom right of the TouchPendant (not the screen) to start the motion tasks again

Errors

During execution, if YuMi is about to reach a collision, detects contact with an external objects that is over a certain threshold, or some of the other integrated safety features are triggered, than an error screen will be displayed on the TouchPendant as shown below in Figure 15 When this happens, the task that triggered the error (most likely the T_ROB_L or T_ROB_R task) will be stopped, but the other tasks will continue to executed.

If this error was triggered by one of the motion tasks, then click the stop button, set the motion task program pointers to their main function, and click the start program to re-execute the motion tasks.

If this error was triggered by one of the state or motion server tasks, then please follow the instructions in the section State or Motion Server Task Error below and follow the instructions listed there before continuing.

4.3.6 Other Relevant Information Regarding YuMi

Enabling Motors

Occasionlly when YuMi is started up, the motors will be set to an off state. To turn the motors on, click the quick access menu button located at the bottom left of the TouchPendant screen. This will open a sidebar on the right of the screen. Click the top-most button (looks likean outlined YuMi) to pull up the Operator Panel. Under the text Motor On/Off, click the on On button to turn the motors on. From this menu, the operator mode can be changed between automatic and manual mode as described

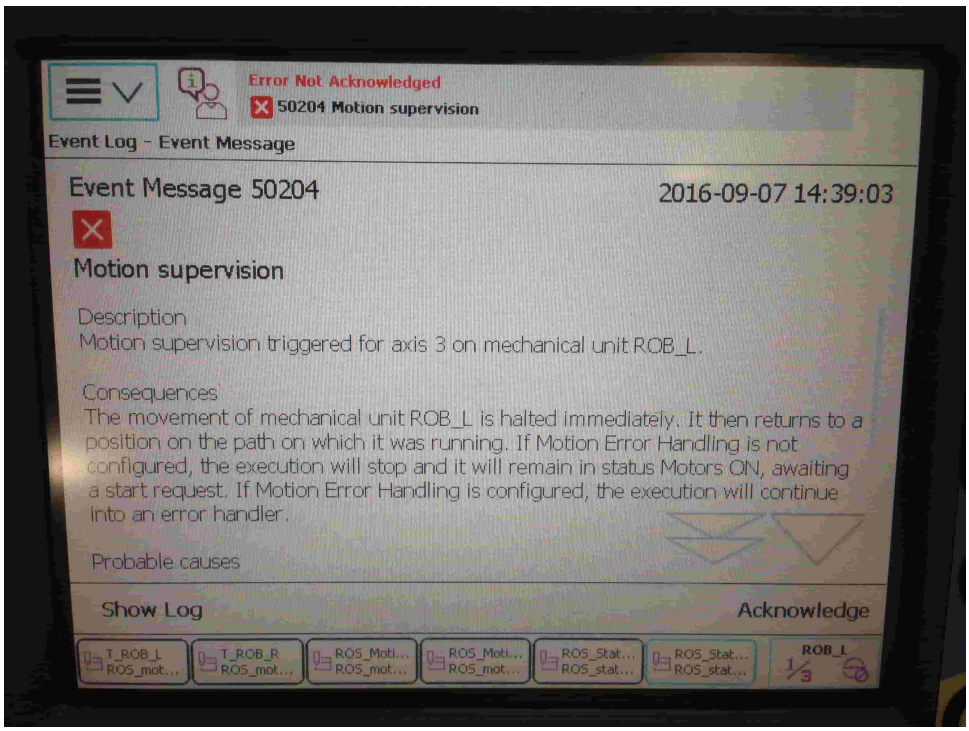


Figure 15: Errors of Teach Pendant

in the section below.

Manual vs Automatic Mode

By default, YuMi is set in Manual mode. In manual mode, the controller limits the max speed of arm movements to 250 millimeters per second (mm/s). Automatic mode allows YuMi to move up to 2 meters per second (m/s). Manual mode should be used for testing code to make sure the paths do not cause any collisions with external objects and to make sure that YuMi moves in the way you expect it to. After the code has been tested, Automatic mode can be used if higher speeds are required for the desired task.

4.4 Control Interfaces

4.4.1 Position Control

Position control through ABB Driver: This interface has a bandwidth of about 10Hz. If you are interested only in driving YuMi arms to certain positions, it might be a good choice. Interface: ROS control, through position *JointTrajectoryControllers*.

Note: additional to this, the parameters MAX_NUM_JOINTS of *industrial_core/simple_message/include/simple_message/joint_data.h* must have a value of 10.

4.4.2 Gripper Control

Currently, YuMi only allows very simple control of the grippers, by setting an effort command in the range [-20, 20].

Gripper State:
If you want to check the grippers position at any moment, run the following command:

```
rostopic echo /yumi/gripper_states
```

ROS Service Interface:
For the ROS service interface the YuMi grippers have been assigned the following IDs:

1. Left gripper: 1
2. Right gripper: 2

Direct ROSService Calls
To open the left gripper:

```
rosservice call /yumi/yumi_gripper/release_grasp \
"gripper_id: 1"
```

To open the right gripper:

```
rosservice call /yumi/yumi_gripper/release_grasp \
"gripper_id: 2"
```

To close the left gripper:

```
rosservice call /yumi/yumi_gripper/do_grasp \
"gripper_id: 1"
```

To close the right gripper:

```
rosservice call /yumi/yumi_gripper/do_grasp \
"gripper_id: 2"
```

Python Service Calls

To close the right gripper:

```
service_proxy = rospy.ServiceProxy
          ("/yumi/yumi_gripper/do_grasp", YumiGrasp)
resp = service_proxy(gripper_id)
#gripper_id is the ID of your gripper, 1 or 2
rospy.sleep(3)
```

To open the right gripper:

```
service_proxy = rospy.ServiceProxy
          ("/yumi/yumi_gripper/release_grasp", YumiGrasp)
resp = service_proxy(gripper_id)
#gripper_id is the ID of your gripper, 1 or 2
rospy.sleep(3)
```

4.5 Live Robot

4.5.1 Starting RAPID scripts (Important)

Everytime you power on the YuMi, or you close the YuMi ROS nodes running on your PC, you need to run the RAPID scripts again (they are Normal tasks inside the IRC5 controller due to safety reasons).

To do that, press the following physical buttons on the FlexPendant:

1. Toggle the Motors On on the FlexPendant by pressing the upper-right corner button that has three horizontal lines.
2. Toggle the Auto mode on the FlexPendant by pressing the upper-right corner button that has two horizontal lines.
3. Move all the program pointers of the RAPID routines to main(), by pressing the upper-right corner button that has one centered horizontal line.
4. Press the Play button located in the bottom-right corner.

You should see and hear the grippers performing three movements: a full opening, a full closing, and then opening again up to 10cm. This is the grippers calibration sequence, and it is advisable to wait for it to finish before running the YuMi ROS nodes.

4.5.2 Live Nodes

Depending on the type of interface (ABB driver interface) installed on YuMi you have to run a different type of node, specified below.

This interface allows position control of the joints by means of a *position_controllers/JointTrajectoryController* per arm.

```
roslaunch yumi_support robot_interface.launch
```

4.5.3 MoveIt!

MoveIt! can take a lot of time to communicate with the action servers of all the possible YuMi joints controllers that we have set up. To decrease the loading time, a good strategy is to search only for the appropriate action servers, related to the control strategy that you want to employ on your robot.

You will need to uncomment some blocks of code in the file *yumi_moveit_config/config/controllers.yaml*, and comment all the others.

The following table describes which blocks you should leave uncommented depending on the type of control interface you want to use:

Control Interface	**Uncomment(Block Name)**
ABB Driver	left_arm, right_arm

Table 8: MoveIt! Changes in Yaml file

Motion Planning user interface in Rviz

To run the MoveIt! server for YuMi, first set up either the Gazebo simulator, or the connection to the real robot.

```
roslaunch yumi_moveit_config demo_online.launch
```

To change the current planning group, open the planning request sub-menu, then click on planning group drop down arrow, and choose the desired planning group.

To run the MoveIt! Setup Assistant, and modify accordingly the YuMi config through the wizard:

```
roslaunch yumi_moveit_config setup_assistant.launch
```

Note: do not overwrite the files *yumi_moveit_controller_manager.launch.xml* and *trajectory_execution.launch.xml*. They are already set up for best performance with YuMi.

5 Miscellaneous

5.1 Calibration Error

This section includes information about when the robot system must be recalibrated. There are two types of calibration, to update the revolution counters or to do a fine calibration.

If the revolution counter memory is lost, the counters must be updated. This will occur when:

- The battery is discharged
- A resolver error occurs
- The signal between a resolver and measurement board is interrupted
- A robot axis is moved with the control system disconnected

The revolution counters must also be updated after the robot and controller are connected at the first installation.
To update the revolution counters is a simple procedure which can be performed using following steps:

1. Manually move the manipulator to the calibration position.
   (a) Release the brakes of the robot arm to be calibrated.
   (b) Move the arm manually so that the synchronization mark (shown in Figure 16) for each joint until it is aligned.

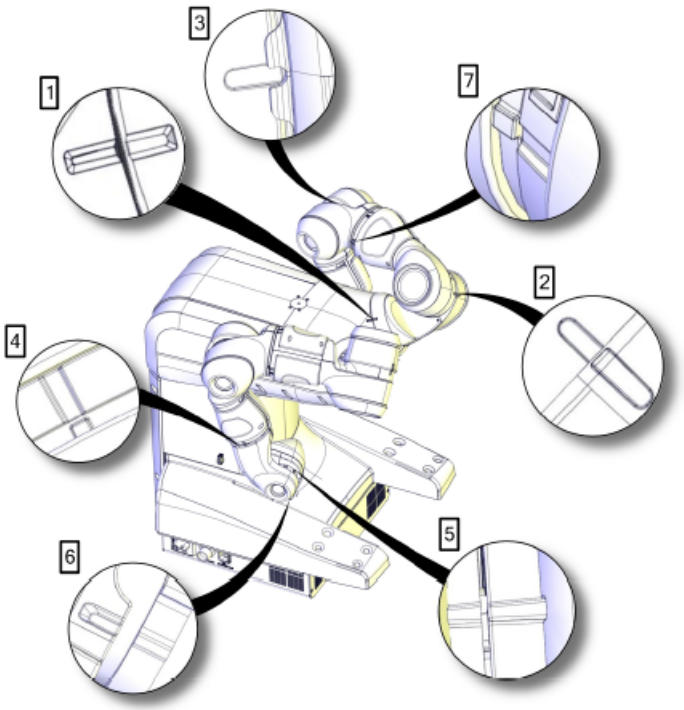


Figure 16: Calibration Scales

   (c) The robot now stands in its calibration position.

2. Select the Calibration with hall sensors (CalHall) routine.
   (a) Open the Program Editor on the FlexPendant.
   (b) Select the task that corresponds to the robot arm to be calibrated. Tap Open.
   (c) If necessary, create a new program. This needs to be done if no existing program is available.
   (d) Select Debug and tap PP to Main.
   (e) Select Debug and tap Call Routine
   (f) Select CalHall.
   (g) Go to Motor On and press the Start button.
3. Select the function Update of revolution counters.
   (a) In the calibration with hall sensors routine (CalHall), tap 2 to select the function of updating the revolution counters.
   (b) Tap to select which joint(s) to update revolution counters for. Joints 1, 2, 3 and 4 are selectable in the first window. Tap Next to open the second window where joints 5, 6 and 7 are selectable.
   (c) Tap OK to start the sampling procedure of the hall sensor. One at a time, each selected joint now rotates back and forth several times to calculate and find the exact position where the hall sensor is aligned with the magnet. Calibration order: axis 1-2-3-4-5-6-7.
   (d) When the sampling of hall sensors is finished, tap OK to confirm.
   (e) If the hall sensor can not be found, error message will be displayed. It means that the joints are not sufficiently aligned according to the synchronization marks. Move the joints so that the synchronization marks are aligned. Tap OK and restart from step 3a.
4. Store the revolution counter setting.

5.2 Calibration Position

The Figure 17 shows the robot in its calibration position.

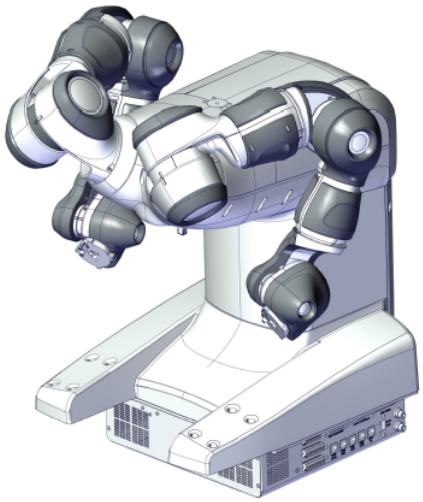


Figure 17: Calibration Position

Exact axis position in degrees in given in Table 9 below.

Axis	IRB 14000 ROB_R	IRB 14000 ROB_R
1	0	0
2	-130	-130
3	30	30
4	0	0
5	40	40
6	0	0
7	-135	135

Table 9: Calibration Position (HOME)

5.3 Operating Modes

The IRB 14000 has two operating modes, Manual mode and Automatic mode. The operator panel is used to switch between Manual mode and Automatic mode. The operator panel is located under the Quickset menu.

5.3.1 Manual Mode

In manual mode the manipulator movement is under manual control. The manual mode is used when programming and for program verification.

5.3.2 Automatic Mode

The automatic mode is the operating mode in which the robot control system operates in accordance with the task program, with functional safeguarding measures. This mode enables controlling the manipulator for example by using the I/O signals on the controller. An input signal may be used to start and stop a RAPID program, another to activate the motors on the manipulator.

6 References

[1] ETH zurich. `https://github.com/ethz-asl/yumi/wiki`.

[2] KTH royal institute of technology. `https://github.com/kth-ros-pkg/yumi/wiki`.

[3] ABB. *Operating manual: FlexPendant.*

[4] ABB. *Operating manual: IRB 14000.*

[5] ABB. *Operating manual: RobotStudio.*

View publication stats